Familiar Guest — Full Solution Design

The complete famguest.com platform: UI surfaces, routing & middleware, and data model

This is the authoritative build-level design for the entire Familiar Guest platform — every user-facing surface, the Next.js routing and middleware architecture, the integration topology, and the full Supabase/Postgres data model. It is scoped around the agreed launch posture: **Trusted-Guest Mode is the launch product**; Public Mode, cross-border payouts, screening, and full tax remittance are designed-in but phased. Each component is tagged LAUNCH (required for first owners to take real bookings) or PHASE 2 (designed now, built after launch).

> **How to read this:** Section 2 is the system at a glance. Sections 3–5 are the three things you asked for — UI, middleware, data model. Section 6 traces the critical flows end-to-end. Section 9 proves coverage against every feature in `CLAUDE.md`. Sections 10–12 cover marketing/SEO/AI-discoverability, first-party analytics, and bot protection/privacy — added per the marketing & security guidance brief.

1 · Launch Posture — What Ships First

Capability	Posture	Rationale
Owner auth (magic link), Gate 1 KYC, property creation, branded booking page, payments + escrow, rental agreement, confirmation email, lodging-tax line items, iCal sync	LAUNCH	The minimum for a founding owner to send a trusted guest a link and get paid safely
Guest CRM (directory + notes), house manual, automated message sequence (email)	LAUNCH	Low-cost, high-trust; differentiators that are cheap to ship
Gate 2 / Public Mode, Verified-Owner badge, booking guarantee	PHASE 2	Trusted-guest is the core use case; public mode is the expansion
Guest screening (Truvi), Protected Booking / damage coverage	PHASE 2	Add-on revenue; not needed for known guests
Multi-currency payout + FX, installments / long-stay	PHASE 2	Launch USD payout; add MXN/CAD + FX spread once volume justifies; gated on MX counsel
Full tax accounting (withholding/remittance, year-end exports), consolidated Pro reporting	PHASE 2	Line-item display ships at launch; remittance is counsel-gated
SMS/WhatsApp (Twilio), AI translation, cloud-photo OAuth, caretaker portal, referrals, one-click rebook	PHASE 2	Email + drag-drop + paste cover launch; richer automation follows
SEO + mobile-first marketing site, AI-agent discoverability (`llms.txt`, structured data)	LAUNCH	Marketing site already exists — these are additive metadata/config, no new surfaces
First-party event capture (page views + clicks, SQL-accessible), Google/Meta pixel support	LAUNCH	One table + one endpoint (§11); cheap to build in now, expensive to retrofit once traffic exists
Bot protection (WAF + Turnstile) + consent management for analytics/pixels	LAUNCH	Security and privacy posture should exist before any public traffic, not after

2 · System Architecture

USERS & SURFACES

Owner
(US / Canada)

Guest
(US / CA / MX / intl)

Caretaker
PHASE 2

Founder / Admin

Public visitor
(marketing)

↓

EXPERIENCE LAYER — NEXT.JS APP ROUTER ON VERCEL (ROUTE GROUPS)

(marketing)
famguest.com landing, waitlist, EN/ES

(owner)
dashboard, listings, calendar, bookings, CRM, money, settings

(guest)
branded booking page, agreement, pay, house manual

(caretaker)
scoped arrivals view

(admin)
verification queue, ops

↓

EDGE & APPLICATION LAYER — MIDDLEWARE + ROUTE HANDLERS + SERVER ACTIONS + VERCEL CRON

middleware.ts
session refresh, role + onboarding gating, locale, bot challenge, anon-id cookie

Server Actions
mutations w/ RLS-scoped server client

Webhook handlers
Stripe · DocuSeal · Truvi · Twilio — signed, idempotent

iCal endpoint
outbound `.ics` per listing

Cron jobs
sync · escrow release · reminders · re-invite

Analytics endpoint
`/api/events` — first-party page-view & click capture (§11)

↓

DATA — SUPABASE (POSTGRES + AUTH + STORAGE), ROW-LEVEL SECURITY ON EVERY TABLE

Postgres
~27 tables across 9 domains (§5)

Auth
email magic-link; roles: owner / caretaker / admin

Storage
property-photos (public) · ownership-docs (private) · signed-agreements (private)

↓

EXTERNAL INTEGRATIONS

Stripe Connect (Custom)
KYC · escrow · payouts · FX · FG subscription billing

Resend
transactional + auth SMTP

DocuSeal
e-signatures

Truvi P2
screening + damage

Twilio P2
SMS / WhatsApp

Anthropic
descriptions · translation · support

OTA iCal
Airbnb / VRBO / Booking

Google & Meta Pixel
consent-gated, server-side CAPI

CROSS-CUTTING — OBSERVABILITY, SECURITY & OPS

Sentry — errors/traces

webhook_events — every event logged + idempotency keys

audit_log — money & verification actions

Alerting — failed payout/webhook → founder

Bot protection — Vercel WAF/Bot Mgmt + Turnstile on public forms (§12)

Design principles: serverless-first; **RLS is the real authorization boundary** (middleware is UX gating only); webhooks always signature-verified + idempotent; money actions never auto-execute outside idempotent, audited bounds; **no SSNs or government IDs stored** (Stripe-hosted KYC); Stripe stays merchant-of-record so FG never custodies funds; all guest-facing copy bilingual EN/ES and accessible.

3 · UI Design

3.1 Five surfaces

Surface	Audience	Auth	Brand
Marketing site	Prospective owners	Public	Familiar Guest brand (the only place it shows)
Owner app	Verified owners	Magic-link session	Familiar Guest (owner's workspace)
Guest booking page	The owner's guests	None — token link	**Owner's property brand** (FG invisible)
Caretaker portal P2	Local helpers	Magic-link, scoped	Neutral
Admin console	Founder	Magic-link, admin role	Internal

3.2 Full site map

```
famguest.com
├─ (marketing)            public · EN/ES · SEO + AI-agent discoverable
│  ├─ /                       landing, value props, pricing, waitlist
│  ├─ /pricing                plan comparison
│  ├─ /legal/{terms,privacy}  bilingual; aviso de privacidad (LFPDPPP)
│  ├─ /sitemap.xml, /robots.txt   generated (§10)
│  └─ /llms.txt               AI-agent service description (§10)
│
├─ (auth)                 public
│  ├─ /login                  email → magic link
│  └─ /auth/callback          code → session, route by status
│
├─ (owner)                session required
│  ├─ /onboarding/profile     name, phone, country, locale, payout currency
│  ├─ /onboarding/verify      GATE 1 — Stripe Connect embedded KYC
│  ├─ /dashboard              home: actions due, upcoming, income snapshot, sync health
│  ├─ /properties             list (status per unit)
│  │  ├─ /new                 create: name, country, GPS*, address
│  │  └─ /[id]
│  │     ├─ /                  overview
│  │     ├─ /content           title, description (paste or AI), photos
│  │     ├─ /pricing           nightly, cleaning(fixed|per-day), fees, deposit, long-stay P2
│  │     ├─ /policies          check-in/out, min/max nights, gaps, cancellation, rules, tax rates
│  │     ├─ /calendar          inbound feeds, outbound .ics URL, blackouts, last-synced
│  │     ├─ /manual            house-manual editor (Baja templates)
│  │     └─ /verify            GATE 2 — ownership doc upload P2
│  ├─ /calendar               all-unit availability
│  ├─ /bookings
│  │  ├─ /                     list + filters
│  │  ├─ /new                  create trusted-guest booking → send link / free booking
│  │  └─ /[id]                 detail: state, line items, agreement, payment, messages
│  ├─ /guests                  CRM directory
│  │  ├─ /[id]                 profile: history, notes, trust tier, re-invite/rebook P2
│  │  └─ /referrals            P2
│  ├─ /money
│  │  ├─ /payouts              balance, escrow holds, payout history, FX P2
│  │  ├─ /income               summaries; consolidated multi-unit Pro / P2
│  │  └─ /taxes                per-booking tax breakdown; year-end exports P2
│  ├─ /messages                conversation inbox (translated) P2
│  └─ /settings
│     ├─ /profile              owner identity, language
│     ├─ /defaults             site parameters (cleaning, fees, deposit, times, rules, tax, currency)
│     ├─ /notifications        channels (email | SMS | WhatsApp) & timing
│     ├─ /plan                 subscription (PAYG/Starter/Host/Pro), card on file, billing
│     └─ /caretakers           invite + scope P2
│
├─ (guest)                public · token · EN/ES · owner-branded
```

```
│   ├─ /book/[token]            listing: photos, desc, map, dates, price, verified badge
│   │   └─ booking flow         dates → details → agreement (DocuSeal) → pay (Stripe) → confirm
│   ├─ /manual/[token]          digital house manual (post-booking)
│   └─ /r/[referralToken]       referral landing P2
│
├─ (caretaker)            scoped session · P2
│   └─ /caretaker               assigned units, arrivals, prep tasks, check-in codes
│
└─ (admin)                admin role
    └─ /admin                   Gate-2 review queue, owners, bookings, webhook log, tax remittance
```

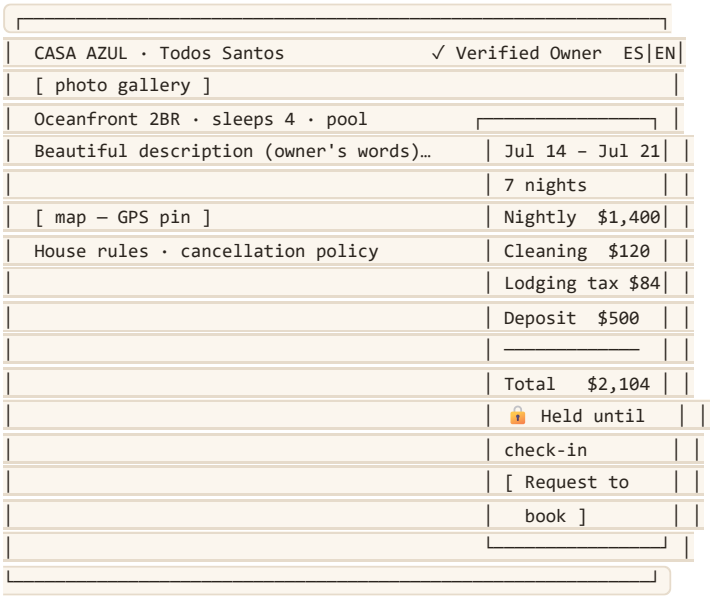
** GPS coordinates are a required field when `country = MX` (street addresses unreliable in Mexico) — used for the check-in Google Maps link and booking-page map.*

3.3 Key screens

Owner — Dashboard home LAUNCH

Cards surface pending owner actions, the next arrival, and an income snapshot. Sync health is always visible (iCal is not real-time — set expectations). Everything is reachable in ≤2 clicks.

Guest — Booking page (owner-branded, no account) LAUNCH



Transparent line-item breakdown (incl. lodging tax + separately-held deposit), escrow reassurance, verified-owner trust mark, bilingual toggle. Works on mobile, no login. Booking flow: **dates → guest details → sign agreement (DocuSeal embed) → pay (Stripe Elements) → confirmation** — guest never sees a Stripe or FG-internal error; all mapped to friendly bilingual messages.

Owner — Property → Pricing/Policies (Site Parameters) LAUNCH

Owner-level defaults set once in `/settings/defaults`, with per-unit overrides here: cleaning fee (**fixed amount OR per-day**), extra-guest/pet fees, damage-deposit default, check-in/out times, min/max nights + gaps + blackout dates, cancellation policy, house rules/quiet hours, lodging-tax rates by jurisdiction, default currency, instant-book vs request-to-book, guest discounts, notification prefs, default language.

Admin — Gate-2 review queue P2

Queue of `ownership_verifications` with `pending` status: document viewer (from private bucket), property address/GPS, approve/reject with notes. Approve flips the property to `public` mode and awards the Verified-Owner badge.

3.4 Design system

- **Tokens** (shared with marketing + docs): Fraunces display, Hanken Grotesk body; forest `#14543F`, terracotta `#C0673E`, cream `#FBF6EE`, ink `#2A241E`, line `#E6DBCB`.
- **Bilingual:** every guest- and owner-facing string in EN/ES via a locale dictionary; `owners.locale` + guest link locale drive default; visible toggle everywhere. Legal/agreement copy is fully localized with country-keyed governing law.
- **Accessibility (hard requirement):** booking page + onboarding keyboard-navigable, screen-reader labelled, WCAG AA contrast — guests include older and non-technical users.
- **Component kit:** shared UI library across owner/guest/admin (buttons, forms, money/line-item display, status pills, date pickers, file upload, document viewer).
- **Mobile-first:** marketing and booking-page layouts designed at mobile breakpoints first, then expanded — most traffic (and most guest bookings) is expected on phones. Performance budgets (Core Web Vitals: LCP/CLS/INP) tracked for `(marketing)` and `(guest)` routes via `next/image`, font subsetting, and SSR.

4 · Routing & Middleware Architecture

4.1 App Router structure

Route groups isolate the surfaces and their layouts. Server Components by default; client components only for interactive widgets (date picker, Stripe/DocuSeal embeds, file upload).

```
app/
├─ (marketing)/          static, ISR; no auth
├─ (auth)/login, auth/callback
├─ (owner)/              layout guards session + Gate-1
├─ (guest)/book, manual  no session; token-scoped data fetch
├─ (caretaker)/          scoped session   · P2
├─ (admin)/              admin-role guard
└─ api/
   ├─ webhooks/stripe        payment, payout, account, subscription
   ├─ webhooks/docuseal      signature completed
   ├─ webhooks/truvi   P2    ├─ webhooks/twilio  P2
   ├─ ical/[token]/route.ts  outbound .ics (GET)
   └─ cron/{sync,escrow,reminders,reinvite}/route.ts
lib/supabase/{client,server,admin}.ts   (already built)
middleware.ts
```

4.2 `middleware.ts` — gating logic

Runs on all routes except static assets and webhooks. Refreshes the Supabase session cookie (via `@supabase/ssr`) and applies UX gating:

Condition	Action
No session + route in `(owner)` / `(caretaker)` / `(admin)`	→ `/login`
Session + `onboarding_status ≠ gate1_complete` + route in `(owner)` non-onboarding	→ `/onboarding/profile` or `/onboarding/verify`
Session + Gate-1 complete + visiting `/onboarding/*`	→ `/dashboard`
Session role ≠ admin + route in `(admin)`	→ 404
Session role = caretaker + route outside `(caretaker)`	→ `/caretaker`
Locale cookie/header	sets request locale for SSR copy
No `fg_aid` (anonymous-id) cookie present	sets a first-party, same-site analytics cookie (§11) — not used for cross-site tracking
Request fails bot heuristic on sensitive routes (`/login`, `/book/*`, `/api/events`)	Vercel Bot Management challenge / Turnstile (§12); flagged requests still served but tagged `is_bot`

> **Authorization vs. gating:** middleware decides *where you can navigate*; it never decides *what data you can read or write*. That is enforced by Postgres RLS using `auth.uid()` and role claims, so a forged request or direct API call still cannot cross tenant boundaries. Service-role access (admin console, webhooks, cron) bypasses RLS and is used only in server-only code paths.

4.3 Route handlers

- **Webhooks** — every handler: (1) verify provider signature against the signing secret, (2) check `webhook_events` for the event id (idempotency), (3) process inside a transaction, (4) record the event + outcome, (5) return 2xx only on success so the provider retries on failure. Guest-facing surfaces never depend on a webhook completing synchronously.
- **iCal outbound** — `GET /api/ical/[token].ics` returns a freshly-generated VCALENDAR of busy blocks for one listing (token unguessable, per-listing). Read-only, cache-friendly.
- **Auth callback** — exchanges the magic-link `code` for a session, then redirects by onboarding status.

- **Analytics capture** — `POST /api/events` (§11): validates a small fixed schema, rate-limited per `fg_aid`, writes via service-role client. Never blocks page render — called fire-and-forget from the client.

4.4 Cron jobs (Vercel Cron → `api/cron/*`, secured by a shared secret header)

Job	Cadence	Does
`sync-calendars`	every 15–30 min	Fetch each inbound iCal feed, upsert external busy blocks, stamp `last_synced`
`release-escrow`	hourly	Release held funds to owner balance on/after check-in (per owner rules); audited
`send-reminders`	hourly	Fire due messages in the sequence (pre-arrival, check-in, mid-stay, checkout, deposit-release)
`reinvite` P2	daily	Season-timed re-invite + post-checkout rebook nudges
`dunning` P2	daily	Retry failed free-booking / subscription charges

5 · Data Model

~27 tables across 9 domains. All tables: `uuid` PK (`gen_random_uuid()`), `created_at` / `updated_at timestamptz default now()`, **RLS enabled**. Default owner policy = rows where the owning `owner_id = auth.uid()`; writes to money/verification columns are service-role only (webhooks/admin). FK = foreign key.

5.1 Domain map

Identity & access
owners · caretakers · caretaker_assignments · admins

Property & listing
properties · listings · listing_photos · listing_settings · pricing_rates · blackout_dates · house_manuals · ownership_verifications

Calendar
calendar_feeds · calendar_blocks

Guests / CRM
guests · referrals

Bookings & money
bookings · booking_line_items · payments · payouts · installments

Trust
agreements · screening_results · protected_bookings

Messaging & manual
message_templates · message_log · conversations · conversation_messages

Tax · billing · ops
tax_records · subscriptions · billing_events · webhook_events · audit_log

Marketing & analytics
analytics_events · consent_log

5.2 Identity & access

`owners` LAUNCH — keyed to `auth.users.id`; created by trigger on signup. `id` (=auth uid) · `email` · `full_name` · `phone` · `country` · `locale` (en|es) · `payout_currency` (USD def) · `onboarding_status` (new|profile_complete|gate1_complete) · `stripe_account_id` · `stripe_onboarding_status` (not_started|pending|verified|restricted) · `stripe_requirements_due` jsonb · `default_settings` jsonb (site parameters)

`caretakers` P2 — `id` (auth uid) · `owner_id` FK · `full_name` · `phone` · `locale` `caretaker_assignments` P2 — `caretaker_id` FK · `property_id` FK · `scope` jsonb (no payments / no full guest list — enforced by RLS) `admins` — `id` (auth uid) · `role` — gates `(admin)` + service-role ops

5.3 Property & listing

`properties` LAUNCH — physical unit + ownership + location. `id` · `owner_id` FK · `name` · `country` (US|MX) · `address` · `latitude` · `longitude` (**required if MX**) · `mode` (trusted_guest|public)

`listings` LAUNCH — the published, bookable representation of a property. `id` · `property_id` FK · `slug` / `booking_token` (unguessable, drives `/book/[token]`) · `ical_token` (outbound feed) · `title` · `description` · `amenities` jsonb · `max_guests` · `bedrooms` · `bathrooms` · `status` (draft|active|paused) · `instant_book` bool

`listing_photos` LAUNCH — `id` · `listing_id` FK · `storage_path` · `position` · `alt` `listing_settings` LAUNCH — per-unit overrides of owner defaults: `cleaning_fee_type` (fixed|per_day) · `cleaning_fee_amount` · `extra_guest_fee` · `pet_fee` · `damage_deposit` · `checkin_time` · `checkout_time` · `min_nights` · `max_nights` · `min_gap_nights` · `cancellation_policy` · `house_rules` · `quiet_hours` · `currency` · `instant_book` `pricing_rates` LAUNCH — `id` · `listing_id` FK · `base_nightly` · `weekend_nightly` · `weekly_rate` P2 · `monthly_rate` P2 · `date_range` (seasonal, nullable) `blackout_dates` LAUNCH — `id` · `listing_id` FK · `start_date` · `end_date` · `reason` `house_manuals` LAUNCH — `id` · `listing_id` FK · `manual_token` (drives `/manual/[token]`) · `content` jsonb (wifi, codes, appliances, parking, rules, local recs) · `locale` `ownership_verifications` P2 (Gate 2) — `id` · `property_id` FK · `owner_id` FK · `document_type` (property_tax_statement|deed|utility_bill|insurance_declaration|fideicomiso|mx_corp_docs) · `storage_path` (private bucket) · `status` (pending|approved|rejected) · `reviewer_notes` · `reviewed_by` · `reviewed_at`

5.4 Calendar

`calendar_feeds` LAUNCH — inbound iCal URLs. `id` · `listing_id` FK · `source` (airbnb|vrbo|booking|other) · `ical_url` · `last_synced_at` · `last_status` (ok|error) · `last_error` `calendar_blocks` LAUNCH — busy blocks (from feeds OR FG bookings, unified availability). `id` · `listing_id` FK · `start_date` · `end_date` · `source` (feed|booking|blackout) · `feed_id` FK? · `booking_id` FK?

5.5 Guests / CRM

`guests` LAUNCH — owner-private directory. `id` · `owner_id` FK · `full_name` · `email` · `phone` · `locale` · `trust_tier` (vetted|returning|referral) · `notes` · `preferences` jsonb · `referred_by_guest_id` FK? `referrals` P2 — `id` · `owner_id` FK · `referrer_guest_id` FK · `referral_token` · `status`

5.6 Bookings & money

`bookings` LAUNCH — central state machine (see §6.2). `id` · `listing_id` FK · `guest_id` FK · `owner_id` FK · `check_in` · `check_out` · `guests_count` · `currency` · `status` (draft|link_sent|pending_agreement|pending_payment|confirmed|checked_in|completed|cancelled|refunded) · `mode` (trusted|public) · `is_free_booking` bool · `total_amount` · `escrow_release_at`

`booking_line_items` LAUNCH — itemized, immutable per booking. `id` · `booking_id` FK · `type` (nightly|cleaning|extra_guest|pet|lodging_tax|damage_deposit|screening|protection|discount) · `label` · `amount` · `currency` `payments` LAUNCH — `id` · `booking_id` FK · `stripe_payment_intent_id` · `amount` · `currency` · `status` (requires_action|processing|held|released|refunded|failed) · `escrow_held_at` · `escrow_released_at` `payouts` LAUNCH — `id` · `owner_id` FK · `booking_id` FK · `stripe_payout_id` · `amount` · `currency` · `fx_rate` P2 · `status` `installments` P2 — long-stay split pay. `id` · `booking_id` FK · `sequence` · `due_date` · `amount` · `stripe_payment_intent_id` · `status`

5.7 Trust

`agreements` LAUNCH — `id` · `booking_id` FK · `docuseal_submission_id` · `template_id` · `locale` · `governing_country` (US|MX) · `status` (sent|signed|declined) · `signed_pdf_path` (private bucket) · `signed_at` `screening_results` P2 — `id` · `booking_id` FK · `truvi_ref` · `status` (pending|pass|review|fail) · `result` jsonb `protected_bookings` P2 — `id` · `booking_id` FK · `coverage_tier` · `truvi_policy_ref` · `premium_amount`

5.8 Messaging & manual

`message_templates` LAUNCH — `id` · `owner_id` FK? (null = system default) · `key` (confirmation|pre_arrival|checkin|mid_stay|checkout|deposit_release) · `locale` · `channel` (email|sms|whatsapp) · `subject` · `body` `message_log` LAUNCH — `id` · `booking_id` FK · `template_key` · `channel` · `to` · `status` (queued|sent|delivered|failed) · `provider_ref` · `sent_at` `conversations` / `conversation_messages` P2 — two-way translated owner↔guest threads. `booking_id` / `guest_id` · `body_original` · `body_translated` · `direction` · `locale`

5.9 Tax · billing · ops

`tax_records` LAUNCH (line items) / remittance P2 — `id` · `booking_id` FK · `jurisdiction` · `tax_type` (us_tot|mx_ish|mx_iva|mx_isr_withholding) · `rate` · `amount_collected` · `remittance_status` (not_applicable|pending|remitted) · `remitted_at` `subscriptions` LAUNCH — FG's billing of the owner. `id` · `owner_id` FK · `plan` (payg|starter|host|pro) · `billing_interval` (monthly|annual) · `stripe_subscription_id` · `status` · `card_on_file` bool (required if free bookings enabled) · `free_month_until` `billing_events` LAUNCH — `id` · `owner_id` FK · `type` (subscription|commission|free_booking_fee|addon) · `amount` · `stripe_ref` · `status` `webhook_events` LAUNCH — `id` · `provider` · `event_id` (unique — idempotency) · `type` · `payload` jsonb · `processed_at` · `status` `audit_log` LAUNCH — `id` · `actor` (owner|admin|system) · `actor_id` · `action` · `entity` · `entity_id` · `metadata` jsonb · `created_at` — every money + verification event.

5.10 Marketing & analytics

`analytics_events` LAUNCH — first-party browse events, one row per page view or click, modeled on the standard GA4/Snowplow event shape (`event_name` + flat context columns + open `properties`). `id` · `occurred_at` · `event_name` (page_view|click|form_submit|booking_step|...) · `anonymous_id` (first-party `fg_aid` cookie, §11) · `session_id` · `user_id` FK? (owner/guest if authenticated) · `user_role` (visitor|owner|guest|caretaker) · `page_path` · `page_url` · `referrer` · `utm_source` ·

`utm_medium` · `utm_campaign` · `device_type` (mobile|tablet|desktop) · `country` (edge geo) · `is_bot` bool · `properties` jsonb (element id/label, booking id, etc.)

`consent_log` LAUNCH — records cookie-consent decisions for privacy compliance. `id` · `anonymous_id` / `user_id` · `category` (analytics|marketing_pixels) · `granted` bool · `region` (eu|uk|ca|us|mx|other) · `recorded_at`

RLS & access: both tables are **insert-only via the service-role `/api/events` endpoint** — the anon key cannot write directly, so client code can't be used to forge arbitrary rows. Owner-scoped reporting (e.g. "views on my listing") is exposed through a Postgres VIEW joining `analytics_events` to the owner's `listings` / `properties`, filtered by `owner_id = auth.uid()`. For ad-hoc SQL access (owner via Supabase SQL editor, Claude via a read-only `DATABASE_URL`), a dedicated read-only Postgres role has `SELECT` on `analytics_events`, `consent_log`, and the reporting views only — no write access, no access to PII-bearing tables.

> **Compliance constraints honored by the schema:** No table stores SSNs, government IDs, or KYC documents — Stripe Connect's embedded components handle that PII directly (hard stop #3). `ownership-documents` and `signed-agreements` live in private, access-controlled buckets. `escrow_release_at` + the hourly cron + `audit_log` implement "never release funds before check-in without owner instruction" (#4). `mode=public` is reachable only via an approved `ownership_verifications` row (#5). `tax_records` separates collection from remittance so MX remittance can be switched on post-counsel (#6, #10).

6 · Key Flows & State Machines

6.1 Owner onboarding (two gates)

Magic-link signup trigger creates `owners` row (new) → **Profile** name, country, locale, currency → profile_complete → **GATE 1 — Stripe KYC** embedded Custom onboarding → webhook sets verified → gate1_complete → **Trusted-Guest Mode** create listings, send links, take payment → **GATE 2 — Ownership** P2 per-property doc → admin approve → mode=public

6.2 Booking lifecycle + escrow LAUNCH

draft owner builds booking → **link_sent** guest opens `/book/[token]` → **pending_agreement** guest signs DocuSeal → webhook → **pending_payment** Stripe PI; deposit held separately → **confirmed** funds HELD in escrow; confirmation email; calendar block created

checked_in at/after check-in, cron releases escrow → owner balance (audited) → **completed** checkout; deposit released after inspection window → **payout** Stripe payout to owner (USD; MXN/CAD+FX P2) → **cancelled / refunded** per cancellation policy; escrow reversed if pre-release

Free bookings skip payment, charge the owner $5 (`billing_events`), still generate the agreement + messages. A failed free-booking charge blocks the booking with an owner-only error (guest never sees it).

6.3 Calendar sync LAUNCH

Inbound: cron fetches each `calendar_feeds.ical_url` every 15–30 min → upserts `calendar_blocks(source=feed)` → stamps `last_synced_at`. Outbound: `/api/ical/[token].ics` serves FG bookings + blackouts as a VCALENDAR for other platforms to import. **Two one-way feeds, not real-time** — owner UI shows last-synced and a double-booking-window caveat.

6.4 Tax handling LAUNCH (display) / P2 (remittance)

On booking creation, applicable `tax_records` are computed from `listing_settings` jurisdiction rates and shown as guest line items (US TOT / MX ISH/IVA). Remittance status starts `pending` / `not_applicable`; actual withholding/remittance to SAT and year-end exports switch on after Mexican counsel confirms FG's platform-of-record obligations. **Handling + reporting, never advice.**

7 · Integrations

Service	Used for	Launch	Notes
Stripe Connect (Custom)	KYC (Gate 1), PaymentIntents, escrow (delayed payout), payouts, FX, FG subscription + free-booking billing	✓	Merchant-of-record; embedded components so no PII stored; MoR avoids money-transmitter licensing
Resend	Transactional email + Supabase auth SMTP	✓	Verify famguest.com domain before launch
DocuSeal	Bilingual rental agreements, e-sign before payment	✓	US ESIGN/UETA · MX Código de Comercio + NOM-151; governing-law clause keyed to property country
Anthropic API	AI descriptions, EN↔ES translation, support deflection	✓ desc	Translation/support deflection P2
OTA iCal	Inbound/outbound calendar	✓	iCal only — **never** scrape Airbnb (hard stop #1)
Truvi	Guest screening + damage protection	P2	Supports MX guest IDs (verify non-US coverage)
Twilio	SMS / WhatsApp messaging	P2	Email-only at launch

Cloud photos (OAuth)	Google/iCloud/Dropbox import	P2	Drag-drop + mobile upload at launch
Google Analytics 4 / Ads	Marketing + SEO/Ads attribution	✓	Free; client-side load gated on `consent_log` (marketing_pixels) + Consent Mode v2 (§12.2)
Meta Pixel + CAPI	Meta ad conversion tracking	P2	Stood up only when Meta ad campaigns run; consent-respecting server-side CAPI fallback (§12.2)
Vercel WAF + BotID	Platform-wide bot & abuse protection	✓	Pro-plan included; WAF free, BotID Deep Analysis metered & used selectively (§12.1)
Cloudflare Turnstile	CAPTCHA on waitlist, login, booking forms	✓	Free widget (no DNS change), privacy-friendly, low-friction

8 · Security & Compliance Mapping

Constraint (CLAUDE.md)	Where enforced
No Airbnb scraping (#1)	iCal-only sync; no OTA fetch beyond owner-pasted `.ics` URLs
No SSN/ID/KYC storage (#3)	Stripe embedded KYC; no PII columns; private buckets for ownership/agreements only
No fund release before check-in (#4)	`escrow_release_at` + hourly cron + `audit_log`; owner override only
No payment before Gate-1 verified (#5)	middleware + RLS + booking creation guard on `stripe_onboarding_status=verified`
Webhooks signed + idempotent + logged (#dev conv)	every handler verifies signature, dedupes on `webhook_events.event_id`
No raw Stripe/Supabase errors to guests	error mapping layer on `(guest)` surfaces
Multi-jurisdiction privacy/e-sign	bilingual legal copy, country-keyed governing law, subprocessor list, PII minimized
Accessibility	WCAG-AA booking + onboarding, keyboard + screen-reader
Malicious/untrusted bot protection (marketing brief)	Vercel WAF + BotID (selective Deep Analysis) + Turnstile on public forms + rate limiting; good-bot allowlist vs default-deny (§10.2, §12.1)
Analytics/pixel privacy compliance (marketing brief)	`consent_log` + cookie banner + Consent Mode v2; first-party-only event capture, no third-party tracking cookies (§12.2)
Booking-token routes not crawlable/indexable (security ⊂ AI-discoverability)	`robots.txt` disallows `(guest)` / `(owner)` / `(admin)` / `(caretaker)`; only `(marketing)` is crawlable (§10.2)

9 · Feature Coverage Check (vs. CLAUDE.md)

Feature in project memory	Covered	Where
Owner-first, trusted-guest, invisible brand	✓	Owner-branded `(guest)` booking page; FG only on marketing
Magic-link auth, Gate-1 KYC, Gate-2 ownership	✓	§4.2, §5.2, §5.3, §6.1
Two booking modes (trusted / public)	✓	`properties.mode`, gated by `ownership_verifications`
Branded booking page, no guest account, mobile, transparent fees	✓	§3.3, `(guest)` routes, `booking_line_items`
Escrow (delayed payout) + separate damage deposit	✓	`payments`, `escrow_release_at`, §6.2
Guest CRM: directory, trust tiers, notes, re-invite, rebook, referral	✓	`guests`, `referrals` (re-invite/rebook P2)
iCal inbound + outbound, last-synced, not-real-time caveat	✓	`calendar_feeds`, `calendar_blocks`, §4.4, §6.3
Rental agreement (DocuSeal), bilingual, signed before payment, stored	✓	`agreements`, §6.2, §7
Automated messaging sequence (email; SMS/WhatsApp P2), bilingual	✓	`message_templates`, `message_log`, cron
Check-in message with Google Maps GPS link	✓	`properties.latitude/longitude` (required MX) → checkin template
Digital house manual, private link	✓	`house_manuals`, `/manual/[token]`
Owner settings / site parameters (cleaning fixed-or-per-day, fees, deposit, times, min/max, blackouts, cancellation, rules, tax rates, currency, instant-book, discounts, notifications, language)	✓	`owners.default_settings`, `listing_settings`, `pricing_rates`, `blackout_dates`
Photo onboarding: drag-drop + mobile (launch); cloud OAuth (P2); AI description	✓	`listing_photos`, `/properties/[id]/content`
Free bookings ($5 owner charge, card-on-file, blocks on fail)	✓	`bookings.is_free_booking`, `subscriptions.card_on_file`, `billing_events`
Cross-border: multi-currency + payouts + FX	✓ P2	`payouts.fx_rate`, `owners.payout_currency`
Long-stay rates + installments	✓ P2	`pricing_rates` weekly/monthly, `installments`
Remote-owner ops: caretaker scoped login + digital check-in	✓ P2	`caretakers`, `caretaker_assignments`, house-manual codes
Rental Income & Tax Accounting (line items launch; remittance + exports P2)	✓	`tax_records`, `/money/taxes`, `/money/income`
Income reporting + consolidated (Pro)	✓	`/money/income` (consolidated = Pro / P2)
Plans (PAYG/Starter/Host/Pro), annual, first-month-free, 5 caretaker seats	✓	`subscriptions`, `/settings/plan`
Verified-Owner badge, booking guarantee	✓ badge / guarantee gated	badge on Gate-2 approve; guarantee deferred until funded reserve
Observability: webhook log, audit, alerting	✓	`webhook_events`, `audit_log`, Sentry
AI-agent service discovery, strong SEO, mobile-first (marketing/security brief)	✓	`/llms.txt`, structured data, sitemap/robots, §10
First-party browse-event capture to SQL DB (owner + Claude access), GA/Meta pixel support (marketing/security brief)	✓	`analytics_events`, `consent_log`, read-only SQL role, §11
Bot/abuse protection + international privacy compliance for analytics (marketing/security brief)	✓	Vercel WAF/Bot Mgmt, Turnstile, `consent_log`, Consent Mode v2, §12

Conclusion: every feature in `CLAUDE.md` maps to a surface, route, and table in this design. Launch-tagged components form a complete Trusted-Guest-Mode product (owner verifies → lists → guest books → signs → pays into escrow → gets messages → owner is paid after check-in, with tax line items and calendar sync). Phase-2 components are designed into the same schema and routing so they extend rather than refactor.

10 · Marketing, SEO & AI-Agent Discoverability

These additions are layered on top of the existing `(marketing)` route group — no new surfaces, just metadata, generated files, and structured data.

10.1 SEO

- **Next.js Metadata API** on every marketing page: title, description, canonical URL, Open Graph + Twitter card images, bilingual `hreflang` (`en` / `es`) alternates.
- `/sitemap.xml` generated via Next.js's `app/sitemap.ts`, covering all `(marketing)` pages in both locales. `/robots.txt` generated via `app/robots.ts`.
- All `(marketing)` pages are server-rendered (already true) — fast TTFB and fully-rendered HTML for crawlers without needing JS execution.
- Semantic HTML (proper heading hierarchy, landmark regions) doubles as both SEO and accessibility groundwork.

10.2 AI-agent discoverability

- `/llms.txt` at the domain root (the emerging convention for AI-agent service discovery): a concise, plain-text description of what Familiar Guest is, who it's for, the plans, and links to `/pricing` and key marketing pages — written for an LLM to summarize accurately when a user asks "what is Familiar Guest / should I use it."
- **Structured data (JSON-LD):** `Organization` + `Service` / `Product` schema on the marketing homepage and `/pricing` (plans as `Offer`s). On `(guest)` booking pages, `LodgingBusiness` schema for the listing (name, address/geo, amenities, price) — improves both search rich-results and agent comprehension of a specific booking page.
- **Trusted-agent allow vs. malicious-bot block — two rules, one mechanism.** The brief asks to *describe the service to trusted AI agents* **and** *guard against malicious bots*; these are handled as a deliberate split:
  - **Allowlist named good crawlers** in `robots.txt` — `Googlebot`, `GPTBot`, `ClaudeBot`, `PerplexityBot`, etc. — scoped to `(marketing)` only. This is the "trusted agent discovery" path.
  - **Default-deny everything else** at the WAF (§12.1): unknown/abusive automated traffic is challenged or blocked. Allowlisting a few good agents is not the same as opening the site to all bots.
- `robots.txt` **scope is deliberate:** `(marketing)` is crawlable by the allowlisted agents and search engines. `(guest)` booking-token routes (`/book/[token]`, `/manual/[token]`), `(owner)`, `(caretaker)`, and `(admin)` are **disallowed** — these contain guest PII and unguessable-but-not-secret tokens that should never be indexed or summarized by a crawler (security consideration, not just SEO).

10.3 Mobile-first

Already a design-system principle (§3.4) — restated here because it's explicitly called out as a requirement: layouts are designed at mobile breakpoints first for `(marketing)` and `(guest)`, since most visitors and guests will be on phones. Performance budgets (Core Web Vitals) are tracked for these route groups.

11 · First-Party Analytics Architecture

This addresses the open design question: how to add first-party browse-event capture without disrupting the existing architecture. **Verdict: it's additive, not structural** — one table, one endpoint, one small client script, and one middleware cookie. Nothing else in §2–§9 changes.

11.1 Capture flow

1. middleware.ts
On first visit, sets a first-party, same-site `fg_aid` cookie (anonymous id) and a `fg_sid` session cookie. No third-party cookies.

→

2. Tiny client script
Loaded in root layout (~1KB, no external domain). Fires `page_view` on navigation and `click` on tagged elements (CTAs, booking-flow steps). **Buffers events and flushes in batches** via `navigator.sendBeacon` on a short timer and on page-hide.

→

3. POST /api/events
Same-origin route handler accepts a **batch** of events per request. Validates a small fixed schema, attaches server-known context (geo, user agent, `auth.uid()` if logged in).

→

4. analytics_events
Written via service-role client (anon key has no insert grant) — prevents arbitrary client-side writes.

11.2 Why this fits the existing design

- **Same Supabase/Postgres instance** — "accessible by Claude and the owner using SQL" is satisfied without standing up a separate analytics stack. A read-only Postgres role gives both SQL access to `analytics_events` + reporting views, nothing else.
- **Same auth/session model** — `fg_aid` / `fg_sid` are additional cookies set by the same `middleware.ts` that already refreshes Supabase sessions; when a visitor logs in, `user_id` / `user_role` get attached to subsequent events, linking anonymous and authenticated activity for that session.
- **Same route-handler pattern** — `/api/events` follows the exact pattern as the webhook handlers (§4.3): signature/shape validation, service-role write, logged.
- **Industry-standard shape** — the `event_name` + flat context + `properties` jsonb shape mirrors GA4/Segment/Snowplow, so if volume later justifies exporting to a dedicated analytics warehouse, the event model translates directly without redesign.
- **Batching keeps cost flat** — each request to `/api/events` is a serverless invocation + DB write, so the client buffers events and flushes them in a single batched `sendBeacon` (on timer + page-hide). This is the same pattern GA4/Snowplow use; it keeps per-event infra cost near zero from launch through scale.

> **Why custom-in-Postgres over PostHog/GA-as-warehouse:** the requirement is a *SQL database the owner and Claude can query*. A table in our own Supabase Postgres satisfies that literally — no second vendor, no per-seat cost, plain SQL access. PostHog's free tier (1M events/mo, HogQL, session replay) is a strong alternative, but its SQL runs against PostHog's warehouse, not the owner's DB, and adds a stack to consent-manage. We keep analytics in-house and revisit PostHog only if we later want session replay/heatmaps.

11.3 Page views vs. clicks

- **Page views**: captured both server-side (middleware can log the initial request) and client-side (for SPA navigations within the App Router) — deduplicated by `session_id` + `page_path` + timestamp window.
- **Clicks**: client-side only, via a small `data-track="..."` attribute convention on key elements (booking CTA, plan selection, waitlist submit) — kept deliberately minimal rather than tracking every click, to keep `properties` meaningful.

11.4 Scaling note PHASE 2

At launch volume, Postgres handles this easily. If/when event volume grows large (thousands of owners, high-traffic marketing pages), consider: partitioning `analytics_events` by month, or offloading raw events to a dedicated store (Tinybird/ClickHouse) while keeping aggregated reporting views in Supabase for Claude/owner SQL access. Not needed for launch — noted so the schema (event-shaped, not relational) doesn't need to change later.

12 · Bot Protection & Privacy / Consent

12.1 Bot protection

Built entirely from features included on the **Vercel Pro plan (~$20/mo) we are already on** — no Enterprise tier, no second CDN hop required.

- **Vercel WAF (free on all plans)** — DDoS mitigation, IP blocking, and custom firewall rules, enabled platform-wide with no app-code changes. The default-deny rule for non-allowlisted bots (§10.2) lives here.
- **Vercel BotID** — basic bot checks are **included**; the metered "Deep Analysis" check ($0.001/check) is applied **selectively to the highest-risk routes only** (`/login` magic-link, booking submission), not site-wide, so per-check cost stays near zero pre-traffic.
- **Turnstile** (Cloudflare's privacy-friendly CAPTCHA, free — used as a widget, no DNS change) on the highest-risk public forms: marketing waitlist signup, `/login` magic-link request, and guest booking submission on `/book/[token]`. Invisible in the common case — only challenges suspicious traffic.
- **Rate limiting** — Vercel WAF rate limiting includes ~1M allowed requests/mo, keyed by IP + `fg_aid`, on magic-link requests, `/api/events`, and booking creation — prevents both abuse and analytics-event flooding from skewing data.
- Requests that fail bot checks are still served (no broken UX) but **tagged** `analytics_events.is_bot = true`, so security/ops can review bot traffic patterns while marketing reporting excludes it.

> **Deliberately NOT adding Cloudflare as a proxy/WAF hop.** Routing the domain through Cloudflare's WAF would require proxying DNS there, which cuts against the decision to keep DNS at Hostinger (for Google Workspace email). Vercel's included firewall + BotID + Turnstile already meets the brief. Cloudflare is held in reserve only if real, sustained abuse appears.

12.2 Consent & privacy compliance

- **Self-built cookie-consent banner** on first visit (bilingual EN/ES), recording the decision to `consent_log` plus a first-party cookie. A small in-house component + `consent_log` satisfies GDPR/LFPDPPP record-keeping for a launch-stage site — **no paid CMP** (Cookiebot/Osano, ~$10–50+/mo) until a customer or regulator requires a certified one. Two categories: *analytics* (first-party `analytics_events` — may run under legitimate-interest in some jurisdictions, but the banner covers it for safety) and *marketing pixels* (Google/Meta).
- **Google Analytics 4 + Consent Mode v2 at launch** (free; needed for SEO/Google Ads attribution): the GA4 pixel loads client-side only after `marketing_pixels` consent, with Consent Mode v2 signaling the choice.
- **Meta Pixel + Conversions API deferred to Phase 2** — stood up only once Meta ad campaigns are actually running. Building server-side CAPI infrastructure speculatively captures no signal pre-spend. When enabled, the same consent-respecting pattern applies: where consent is declined, conversion signal (e.g., booking completed) is sent **server-side and cookieless** via webhook → Meta CAPI / GA4 Measurement Protocol.
- This extends the existing multi-jurisdiction privacy posture (§8, LFPDPPP/GDPR/CCPA/PIPEDA) rather than adding a new compliance surface — `consent_log` is simply the record-keeping piece those frameworks require for analytics/advertising cookies specifically.